



Filière Master : Ingénierie Electrique et Systèmes Embarqués (IESE)

Technologies Web

Le Langage PHP

Pr. Abdelali Boushaba

Année universitaire : 2025-2026

Plan

- Chapitre 1 : Les concepts
- Chapitre 2 : Le langage HTML
- Chapitre 3 : Le langage CSS
- Chapitre 4 : Le langage JavaScript
- Chapitre 5 : Le langage PHP

Chapitre 5 :Le langage PHP

Master/IESE

3

Plan

- Introduction
- Principe de fonctionnement de PHP
- Éléments de base du Langage PHP
- Chaîne de caractère
- Fonctions
- Tableaux
- Formulaires et PHP
- SGBD et PHP

Master/IESE

4

Introduction

Introduction

- PHP est l'acronyme récursif de : Hypertext Preprocessor
- PHP est un langage de script côté serveur
- PHP est un langage de création des sites Web dynamiques
- PHP est multi plate-forme (UNIX / Windows...)
- PHP est un langage interprété comme JavaScript, Python, Perl, etc.
- Le code PHP peut être intégré avec le code HTML
 - Dans le même fichier, on peut trouver du code HTML et PHP
 - Open Source
- PHP est utilisé par plus que 80% des serveurs Web au niveau du monde
- La version actuelle de PHP est la version 8.5

Les points forts de PHP

- ☐ Langage performant :
 - ☐ Rapide
 - ☐ Stable: Pas de blockage
 - ☐ Scalable : son comportement est normal même si le nombre de visiteurs augmente
- ☐ Langage souple
- ☐ Facile à prendre en main
- ☐ Dispose de nombreux outils et d'une très large communauté.
- ☐ multi plateforme :
 - ☐ Multi-OS (Windows, Unix, Linux, etc.)
 - ☐ Multi-serveur HTTP (Apache, IIS, Nginx, etc.)
- ☐ Une interface pour la manipulation de la majorité des SGBD (MySQL, PostgreSQL, MariaDB, Oracle, SQL Server, etc.)
- ☐ Il possède des bibliothèques riches qui permettent d'étendre ses fonctionnalités
- ☐ Plusieurs scripts PHP gratuits sur le web

Master IESE

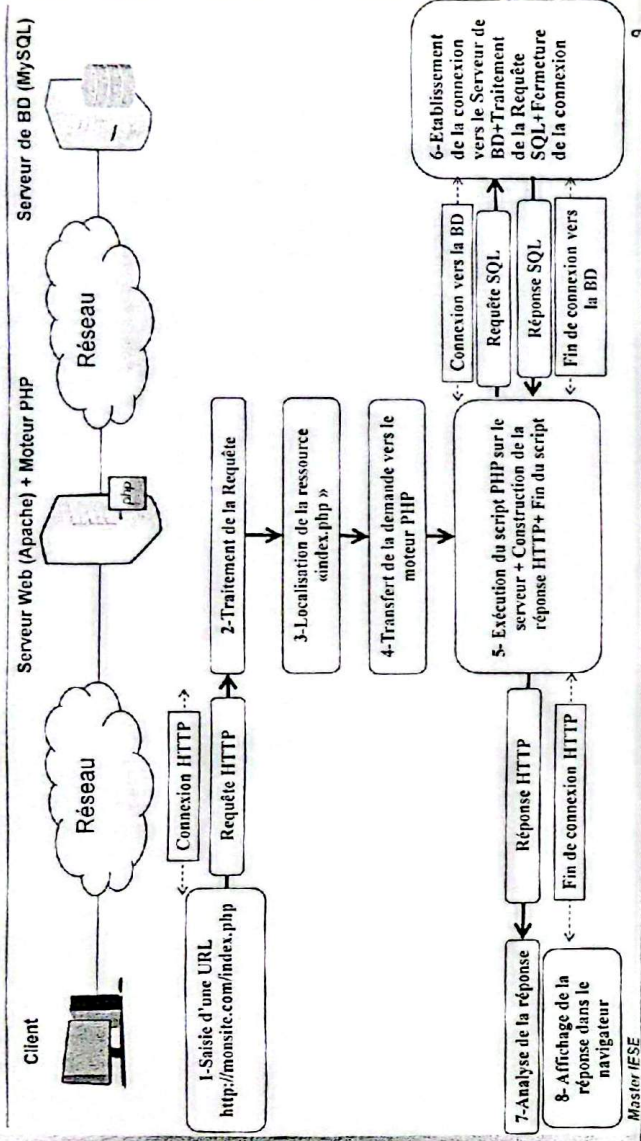
7

Principe de fonctionnement de PHP

Master IESE

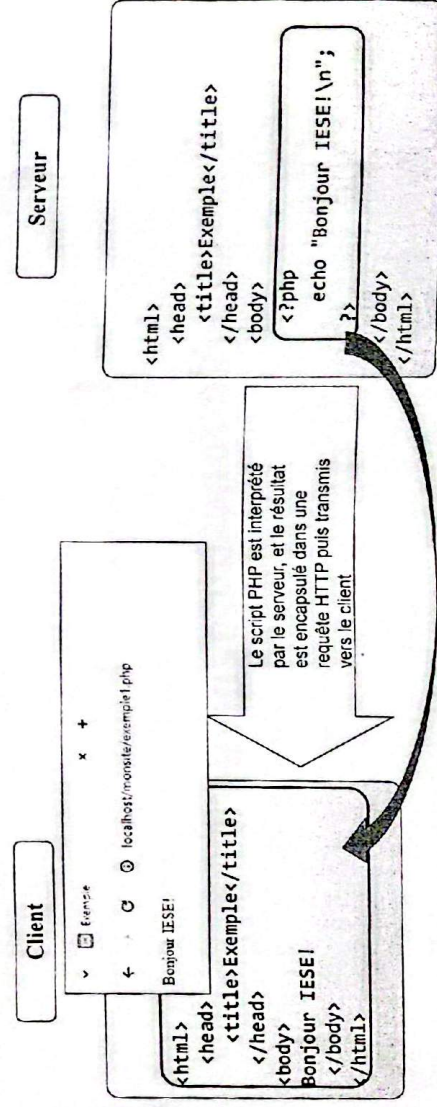
8

Principe de fonctionnement de PHP



9

Principe de fonctionnement de PHP : Exemple



Remarque : Le script PHP est inaccessible par le client

Master IESE

10

Intégration PHP et HTML (1)

- Les scripts PHP sont généralement intégrés dans le code HTML d'une page Web
- L'intégration se fait par plusieurs manières à travers les balises suivantes:

```
<? php  
Code PHP  
?>
```

```
<?  
Code PHP  
?>
```

```
<script language="php">  
Code PHP  
</script>
```

Les fichiers HTML contenant du code PHP ont l'extension .php

Master/ISE

11

Intégration PHP et HTML

- Intégration directe dans le code HTML

```
<?php  
//code PHP  
?>  
<html>  
  <head> <title> Un Exemple </title> </head>  
  <body>  
    //code HTML  
    <?php  
      //code PHP  
    ?>  
  </body>  
</html>
```

Master/ISE

12

Intégration PHP et HTML

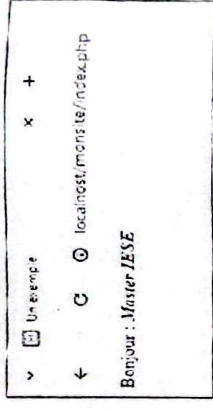
- 1 Inclure un fichier PHP dans un fichier HTML : fonction PHP include()

```
<?php  
echo "<i><b>Master IESE</b></i>";  
?>
```

Fichier message.php

```
<html>  
<head>  
<title> Un exemple </title>  
</head>  
<body>  
<?php  
echo "Bonjour : ";  
include ("message.php");  
?>  
</body>  
</html>
```

Fichier index.php



Éléments de base du Langage PHP

Les fonctions echo, print et printf

- Les fonctions echo , print et printf permettent d'envoyer vers le navigateur des messages et du code HTML
- La fonction echo :
 - Syntaxe:
 - echo "Chaine de caracteres";
 - echo expression ;
 - Exemple
 - echo "Bonjour";
 - echo "Bonjour";
 - \$nom="Ali"; echo "Bonjour \$nom";
 - echo 2*(1+5);
- La fonction print:
 - Syntaxe:
 - print ("Chaine de caracteres");
 - print (expression);
 - La fonction printf : (syntaxe similaire à celle du langage C):
 - Exemple
 - printf ("Le prix TTC est %f",\$pttc);
- La fonction print_r pour afficher les variables complexes (objets)

Master IJSE

15

Syntaxe de base

- Chaque instruction PHP doit être terminée par un point-virgule
- PHP est sensible à la casse
- Les commentaires
 - /* un commentaire
sur plusieurs lignes */
 - // un commentaire sur une ligne
 - # un commentaire sur une ligne

Master IJSE

15

Les variables

- Une variable :
 - commence toujours par le caractère **\$**
 - \$** doit être suivi par une lettre ou le caractère **_** (peut contenir des nombres, mais pas au début)
 - est sensible à la casse
 - peut ne pas être déclaré ni initialiser
 - peut recevoir une valeur par l'utilisation de l'opérateur **=** d'affectation
 - peut adapter son type selon le contexte (typage dynamique)

■ Exemple:

```
$x=10;  
$tva=0.20  
$produit="Fraise";  
$prix=10;  
$x="10";
```

Master I/EE

17

Les types de données

- Les types en PHP sont :
 - Numérique entier : **15** ou réel : **10.5**
 - Chaîne de caractères: **"Bonjour"** ou **'Bonjour'**
 - Booléen: **true** ou **false**
 - Tableau
 - NULL**
 - Objet
 - Etc.

Master I/EE

18

Typage dynamique

En PHP, pas besoin de déclarer préalablement les variables

```
$x = 10.75; //x est de type float
```

```
$x = 10; //maintenant x est de type int
```

```
$x = array(); //maintenant x est un tableau
```

```
$x = '20'; //maintenant x est de type chaîne de caractères
```

isset() permet de vérifier est ce qu'une variable est définie

```
$a;
```

```
isset($a); // retourne true
```

empty() permet de vérifier est qu'une variable n'est pas vide

```
$b=10;
```

```
empty($b); // retourne false
```

19

Typage dynamique selon le contexte

```
$var1 = 10.5;
```

```
$var2 = 5;
```

```
$var3 = "20";
```

```
$var4 = 'Bonjour';
```

```
$som = $var1 + $var2
```

+ \$var3

+ \$var4 ;

Utilisé comme float

Utilisé comme float

Utilisé comme float

Utilisé comme float

10.5

5

"20"

'Bonjour'

15.5

20.0

0.0

25.5

25.5

```
echo $som; // affiche 25.5 qui est un Réel
```

Master I/SE

20

Modification de type

- Le type de la variable peut être modifier par le casting: cast

Syntaxe:

```
$variable = (nom_du_type) valeur
```

Exemples:

```
$x = (integer) '10'; //La variable x est de type int
```

```
$y = (string) 12; //la variable y est de type chaîne de caractères
```

```
$z = (float) $y; //La variable z est de type float avec comme  
//valeur 12
```

Définition des constantes

- On peut définir une constante par différentes manières:

```
define('NOM_CONST', valeur)  
const NOM_CONST= valeur;
```

Exemple:

```
const FILIERE = 'Master IESSE';  
define('PI', 3.1415)  
$R=10;  
$S=$R*$R*PI; //Pas besoin de $ pour accéder à la valeur de la constante PI  
echo "Surface Cercle est $S";
```

Les chaînes de caractères

Chaînes de caractères

■ Interprétation des variables dans les chaînes

Guillemets doubles

```
$x="Master IESE";
```

```
$y="Bonjour $x"; // le contenu de y est «Bonjour Master IESE»
```

■ Guillemets simples

```
$x='Master IESE';
```

```
$y='Bonjour $x'; // le contenu de y est «Bonjour $x»
```

Concaténation de chaînes

La concaténation de deux ou plusieurs chaînes est réalisée par l'opérateur point : « . »

Exemples:

```
$x = 'Bonjour' . 'Master IESE';
```

→ Le contenu de x est la chaîne 'Bonjour Master IESE'

```
$n = 29;
```

```
$y = 'Vous êtes' . $n . ' étudiants';
```

→ Le contenu de y est la chaîne 'Vous êtes 29 étudiants'

Caractères d'une chaîne

Pour accéder à un caractère particulier d'une chaîne **ch**, il faut utiliser les crochets comme pour un tableau. Le premier caractère se trouve en position 0. Le dernier caractère se trouve en position `strlen($ch)-1`.

Exemple

```
$s="Bonjour";
```

```
for ($i=0; $i<strlen($s); $i++)
```

```
    echo $s[$i] . " "; // affiche "B o n j o u r"
```

```
$s[0]='b'
```

```
for ($i=0; $i<strlen($s); $i++)
```

```
    echo $s[$i] . " "; // affiche "b o n j o u r"
```


Quelques onctions de manipulations des chaînes

- int strlen(string \$s) : longueur de s
- int strcmp(string \$s1, string \$s2) : compare les chaînes (comparaison sensible à la casse)
- int strcasecmp(string \$s1, string \$s2) : compare les chaînes (comparaison insensible à la casse)
- strtolower() : retourne une chaîne donnée en minuscule
- strtoupper() : retourne une chaîne donnée en majuscule
- int strpos(string \$ch, string \$sch) : position du premier sous-chaîne sch dans s
- int strrpos(string \$s, string \$sch) : position du dernier sous-chaîne sch dans s

Master I/EEF

27

Les opérateurs

Les opérateurs arithmétiques

\$a + \$b	Somme
\$a - \$b	Différence
\$a * \$b	Multiplication
\$a / \$b	Division
\$a % \$b	Modulo (Reste de la division entière)
\$a ** \$b	Résultat de l'élevation de \$a à la puissance \$b.

Les opérateurs d'incrémentation/décrémentation

\$a++	Retourne la valeur de \$a puis augmente la valeur de \$a de 1
++\$a	Augmente la valeur de \$a de 1 puis retourne la nouvelle valeur de \$a
\$a--	Retourne la valeur de \$a puis diminue la valeur de \$a de 1
--\$a	Diminue la valeur de \$a de 1 puis retourne la nouvelle valeur de \$a

Master I/EEF

28

Les opérateurs de comparaison

\$a == \$b	Vrai si égalité entre les valeurs de \$a et \$b
\$a != \$b	Vrai si différence entre les valeurs de \$a et \$b
\$a < \$b	Vrai si \$a inférieur à \$b
\$a > \$b	Vrai si \$a supérieur à \$b
\$a <= \$b	Vrai si \$a inférieur ou égal à \$b
\$a >= \$b	Vrai si \$a supérieur ou égal à \$b
\$a === \$b	Vrai si \$a et \$b identiques (valeur et type)
\$a !== \$b	Vrai si \$a et \$b différents (valeur ou type)

Les opérateurs logiques

Expr1 and Expr2	Vrai si Expr1 et Expr2 sont vraies
Expr1 && Expr2	Idem
Expr1 or Expr2	Vrai si Expr1 ou Expr2 sont vraies
Expr1 Expr2	Idem
Expr1 xor Expr2	Vrai si Expr1 ou Expr2 sont vraies mais pas les deux
! Expr1	Vrai si Expr1 est non vraie

Structures de contrôle et choix multiples

■ Pas de différences avec celles de C/C++/JavaScript:

```
if  
if-else  
switch
```

■ PHP définit une alternative de if-else imbriquée:

```
if( condition(s)1 )  
    <instruction(s)1>;  
elseif ( condition(s) 2 )  
    <instruction(s)2>;  
else  
    <instruction(s)3>;
```

Master I/EESE

29

Les boucles

■ Pas de différences avec celles de C/C++/JavaScript:

```
for  
while  
do-while
```

■ De même pour les instructions break et continue

■ PHP définit une alternative de for:

foreach → permet de parcourir les tableaux d'une manière simple

Exemple:

```
foreach ($tableau as $valeur) {  
    <instruction (s) utilisant $valeur >;  
}
```

Master I/EESE

30

Les fonctions

Les fonctions

❏ fonction qui ne retourne rien:

Définition :

```
function nom_fonction($param1,$param2,...)
```

```
{
```

```
    <instruction(s)>;
```

```
}
```

Appel de la fonction

```
nom_fonction($a,$b,...);
```

❏ fonction qui retourne une valeur:

Définition :

```
function nom_fonction($param1,$param2,...)
```

```
{
```

```
    <instruction(s)>;
```

```
    return $valeur;
```

```
}
```

Appel de la fonction

```
$r = nom_fonction($a,$b,...);
```


Les fonctions

fonction factorielle

```
<?php
function factorielle($n)
{
    $f=1;
    for($i=1;$i<=$n;$i++)
        $f=$f*$i;
    return($f);
}

$nbr=4;
$r=factorielle($nbr);
print($r);
?>
```

Master I/SE

33

Les fonctions : Variables locales et globales

- Une variable locale à une fonction n'est pas visible de l'extérieur (non accessible de l'extérieur)
- Une variable globale est accessible à partir de n'importe quelle fonction du même fichier
- Syntaxe de déclaration d'une variable globale :
global \$nom_variable;

Master I/SE

34

Les tableaux

Tableaux

- Un tableau en PHP est une structure qui peut contenir plusieurs valeurs ou de tableaux normaux
- Une variable tableau est de type `array`
- La taille d'un tableau est dynamique
- PHP définit deux types de tableaux :

Tableaux scalaires : tableaux normaux

- Les éléments du tableau sont référencés par des indices entiers
- L'accès à un élément s'effectue à travers son indice `i` : `$T[i] = valeur`;

Indices →	0	1	2	3
Valeurs →	Elem1	Elem2	Elem3	Elem4

Tableaux associatifs

- Chaque élément du tableau est référencé explicitement par un indice appelé clé
- La clé peut être de type chaîne de caractères ou numérique
- L'accès à un élément s'effectue à travers sa clé `cle` : `$T[cle] = valeur`;

clés →	cle1	cle2	cle3	cle4
Valeurs →	Elem1	Elem2	Elem3	Elem4

Tableaux scalaires

- 4 Un tableau scalaire peut être initialisé avec différentes manières:

```
$tab = array(val1, val2, val3, ...);  
$tab[0] = val1; $tab[1] = val2; $tab[2] = val3; ...  
$tab[] = val1; $tab[] = val2; $tab[] = val3; ...
```

- 5 Exemples :

```
$Couleurs= array('Bleu', 'Vert', 'Orange', 'Blanc');  
$tab = array('Voiture', 2024, $Couleurs[3]);
```

```
$villes[0] = "Fès";  
$villes[1] = "Rabat";  
$villes[2] = "Casa";
```

```
$prenoms[] = "Ali";  
$prenoms[] = "Kamal";  
$prenoms[] = "Said";
```

- 6 L'accès à un élément du tableau se fait à partir de son indice

Premier indice : 0

dernier indice : `taille_du_tableau -1`

Exemple : `echo $tab[9];` // pour accéder au 10^{ème} élément (echo "\$tab[9]"; incorrect pas de " ")

Master I/SE

37

Affectation des tableaux

- 7 \$tab1 = \$tab2; //Permet de copier tous les éléments de \$tab2 dans \$tab1

- 8 Exemple:

```
<?php
```

```
$Couleurs= array('Bleu', 'Vert', 'Orange', 'Blanc');
```

```
$Couleur_copie = $Couleurs;
```

```
print(" affichage de la copie du tableau: <p>");
```

```
print_r($Couleur_copie );
```

```
?>
```

Master I/SE

38

Parcourir un Tableau

```
1 Soit le tableau tab : $tab= array('Bleu', 'Vert', 'Orange', 'Blanc');
2 Méthode 1:
3 for($i=0; $i < count($tab); $i++) { // count() retourne le nombre d'éléments
4     echo $tab[$i]. " ";
5 }
6 Ou bien:
7 $i=0;
8 while($i < count($tab)) {
9     echo $tab[$i]. ' ';
10    $i++;
11 }
12 //affiche : Bleu Vert Orange Blanc
```

```
1 Méthode 2:
2 foreach($tab as $elem) {
3     echo $elem. " ";
4 }
```

Pour chaque itération de la boucle foreach, la variable \$elem prend successivement une valeur du tableau \$tab.

39

Insersion d'un élément dans un tableau

```
1 array_push($tab,$elem) : insérer un élément $elem à la fin du tableau $tab (Empiler)
2 array_unshift($tab,$elem) : insérer un élément $elem au début du tableau $tab (Emfiler)
```

Exemple:

```
<?php
$Couleurs= array('Bleu', 'Vert', 'Orange', 'Blanc');
print_r($Couleurs);
print("<br> insertion à la fin du tableau<br>");

$Couleurs[4]="Rouge";
$Couleurs[]="Marron";
array_push($couleurs,"Noir");

print_r($couleurs);

print("<br> insertion au début du tableau <br>");
array_unshift($Couleurs,"Violet");
print_r($Couleurs);
```

Master ISE

40

Supression d'un élément dans un tableau

- ▣ `array_pop($tab)` : Retourne et supprime le dernier élément du tableau (Dépiler)
- ▣ `array_shift($tab)` : Retourne et supprime le premier élément du tableau (Défiler)

Exemple:

```
<?php
$Couleurs= array('Bleu', 'Vert', 'Orange', 'Blanc');
print_r($couleurs);
$derniere_Couleur=array_pop($Couleurs);
$premiere_couleur=array_shift($Couleurs);
print("<p>dernière couleur :$derniere_couleur");
print("<p>première couleur :$premiere_couleur");
print("<p>le contenu du tableau après la suppression du premier et du dernier éléments : ");
print_r($Couleurs);
```

?>

Master I/EE

41

Autres fonctions de manipulation des tableaux

- ▣ `print_r($tab)` : Affiche un tableau sans boucle
- ▣ `count($tab)` ou `sizeof` : retournent le nombre d'éléments du tableau
- ▣ `in_array($var,$tab)` : vérifie si la valeur de `$var` existe dans le tableau `$tab`
- ▣ `list($var1,$var2...)` : transforme une liste de variables en tableau
- ▣ `range($i,$j)` : retourne un tableau contenant un intervalle de valeurs
- ▣ `shuffle($tab)` : mélange les éléments d'un tableau
- ▣ `sort($tab)` : trie alphanumérique les éléments du tableau
- ▣ `rsort($tab)` : trie alphanumérique inverse les éléments du tableau
- ▣ `ksort($tab)` : trie alphanumérique les clés du tableau
- ▣ `implode($sep,$tab)` ou `join` : retourne une chaîne de caractères contenant les éléments du tableau `$tab` séparés par chaîne de jointure `$str`
- ▣ `explode($sep,$str)` : retourne un tableau dont les éléments résultent de la chaîne `$str` et qui sont séparés par le séparateur `$sep`
- ▣ `array_merge($tab1,$tab2,$tab3...)` : concatène les tableaux passés en arguments
- ▣ `array_rand($tab)` : retourne un élément du tableau au hasard

Master I/EE

42

Tableaux associatifs

Un tableau associatif peut être initialisé avec différentes manières:

- `$tab = array(cle1 =>val1, cle2 =>val2, cle3 =>val3, ...);`
- `$tab[cle1] = val1; $tab[cle2] = val2; $tab[cle3] = val3; ...`

Exemple 1 :

```
$personne = array("Nom" => "Alami", "Prénom" => "Ali");
```

// qui est équivalent à:

```
$personne["Nom"] = "Alami"; //la clé "Nom" est associée la valeur "Alami".
```

```
$personne["Prénom"] = "Ali";
```

Exemple 2 :

```
$voiture= array("marque" => "Peugeot", "couleur" =>"Bleue", "model" => 2025, matricule => "2525-A-25");
```

clés →	"Nom"	"Prénom"
Valeurs →	"Alami"	"Ali"

clés →	"marque"	"couleur"	"model"	"matricule"
Valeurs →	"Peugeot"	"Bleue"	2025	"2525-A-25"

Master I.ESE

43

Parcourir un tableau associatif

Soit le tableau associatif suivant:

```
$voiture= array("marque" => "Peugeot", "couleur" =>"Bleue", "model" => 2025, matricule => "2525-A-25");
```

Méthode 1: accéder directement aux éléments du tableau sans passer par les clés.

```
foreach($voiture as $selem) {  
    echo "$selem" . " ";  
}  
// affiche : Peugeot Bleue 2025 2525-A-25
```

Méthode 2: accéder simultanément aux clés et aux éléments.

```
foreach($voiture as $cle => $selem) {  
    echo "$cle : $selem" . " | ";  
}  
// affiche : marque : Peugeot | couleur : Bleue | model : 2025 | matricule : 2525-A-25
```

Master I.ESE

44

Quelques fonctions de manipulation des tableaux associatifs

- `array_count_values($tab)` : retourne un tableau associatif contenant les valeurs du tableau `$tab` comme clés et leurs fréquences comme valeur (utile pour évaluer les redondances)
- `array_keys($tab)` : retourne un tableau contenant les clés du tableau associatif `$tab`
- `array_values($tab)` : retourne un tableau contenant les valeurs du tableau associatif `$tab`
- `array_key_exists($cle,$tab)` : vérifie si la cle `$cle` existe dans le tableau `$tab`
- `array_search($val,$tab)` : retourne la clé associée à la valeur `$val`
- `unset($tab[cle])` : supprime et retourne la valeur indexée par la clé `cle`
- `extract($tab)` : permet d'extraire d'un tableau toutes les valeurs, chaque valeur est recopiée dans une variable ayant pour nom la valeur de la clé.

Exemple:

```
$tab = array("nom"=>"Alami", "note"=>15);  
extract($tab);  
echo "$nom $note"; # affiche : Alami 15
```

Master I/ESF

45

Quelques fonctions pour parcourir les tableaux (scalaires ou associatifs) :

- Quelques fonctions alternatives pour le parcours de tableaux (normaux ou associatifs) :

- `reset($tab)` : place le pointeur sur le premier élément
- `current($tab)` : retourne la valeur de l'élément courant
- `next($tab)` : place le pointeur sur l'élément suivant
- `prev($tab)` : place le pointeur sur l'élément précédent
- `end($tab)` : place le pointeur sur le dernier élément
- `each($tab)` : retourne la paire clé/valeur courante et avance le pointeur

Exemple :

```
$tab= array('Bleu', 'Vert', 'Orange', 'Blanc');  
$n = count($tab);  
reset($tab);  
for($i=1; $i<=$n; $i++) {  
    echo current($tab)." ";  
    next($tab);  
}
```

Master I/ESF

46

Exemple

```

?php
$tab=array("Master","IESE",2025);
$n = count($tab);
echo "Taille du tableau : " . $n; // Affiche: 3
echo "<br>";

if(!in_array("IESE",$tab))
    echo "Existe";
else
    echo "N'existe pas";
// Affiche: Existe

list($a,$b,$c)=$tab;
echo "<br>";
echo "Sa | $b | $c"; // Affiche: Master | IESE | 2025

$chaine1 = implode(" - ", $tab);
echo "<br>";
echo "Chaine1"; // Affiche: Master - IESE - 2025

$tab=explode(" - ", $chaine1);
$chaine2=implode(" * ", $tab);
echo "<br>";
echo "Chaine2"; // Affiche: Master * IESE * 2025
?>

```

Master/IESE

47

← ↻ ↺ localhost/monsite/chaine.php

Taille du tableau : 3

Existe

Master | IESE | 2025

Master - IESE - 2025

Master * IESE * 2025



Tableaux Multi-dimensions

■ L'élément d'un tableau peut être un autre tableau → on parle de tableau multi-dimensions

■ Exemple

```

$tab1= array("nicoice", "mexicaine", "variee");
$tab2= array("poisson", "tagine", "brochettes");
$tab3 = array("banane", "pomme", "orange");
$menu = array("entree" => $tab1, "plat" => $tab2, "dessert" => $tab2);

```

// qui est équivalent à:

```

$menu = array("entree" => array("nicoice", "marocaine", "variee"),
              "plat" => array("poisson", "tagine", "brochettes"),
              "dessert" => array("banane", "pomme", "orange"));

```

//affichage du tableau multi-dimensions menu:
print_r(\$menu);

Master/IESE

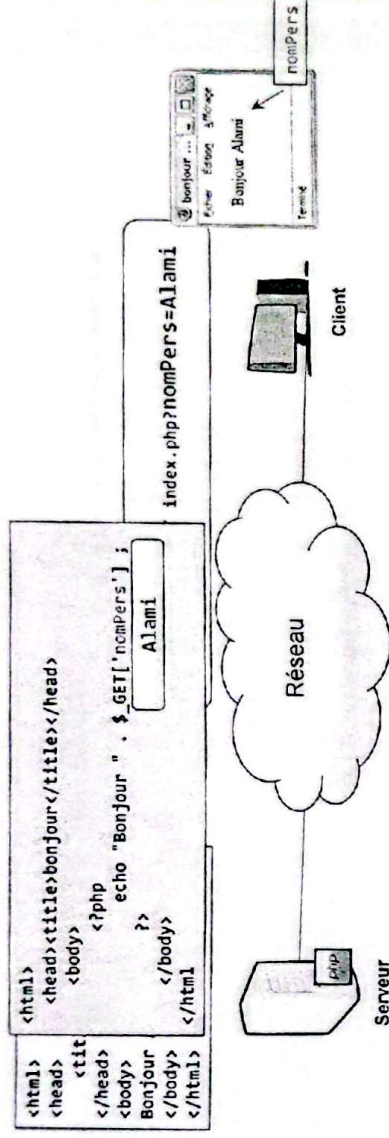
48

Formulaires et PHP

Traitement des données de formulaires par PHP

- PHP permet de traiter les données saisies grâce à un formulaire HTML.
- Dans ce cas, l'attribut `ACTION` du formulaire doit contenir le chemin du fichier PHP à exécuter par le serveur Web.
- Après récupération par le serveur Web, les données sont contenues dans l'une des variables superglobales de type tableau associatif `$_GET` ou `$_POST` selon la méthode de la requête (ou `$_REQUEST` qui reçoit le contenu de `$_GET` et `$_POST`).
- La valeur peut être récupérée par une clé qui porte le même nom que le champ du formulaire de la page HTML de saisie.

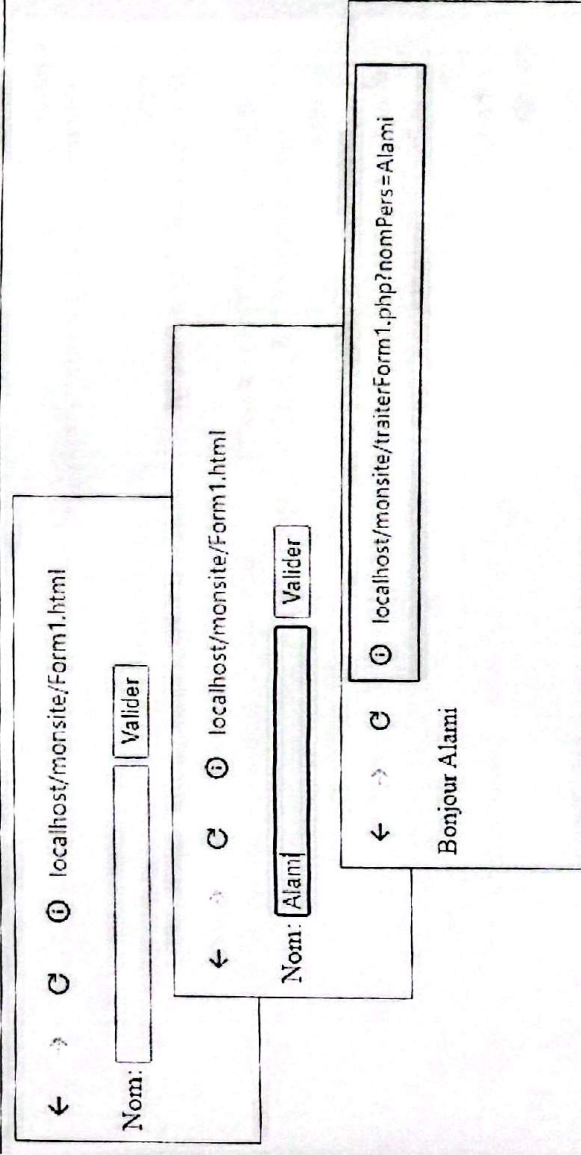
Traitement des données de formulaires



Master I/EEF

51

Formulaire : Zone de saisie



Master I/EEF

52

Exemple – Formulaire HTML

```
<html>
<head>
  <title>Traitement des formulaires</title>
</head>
<body>
  <form action="traiterForm1.php" method="get">
    Nom: <input type="text" name="nomPers">
    <input type="submit" value="Valider">
  </form>
</body>
</html>
```

Master I/EE

53

Exemple 1 – Traitement en PHP

```
<html>
<head><title>bonjour</title></head>
<body>
  <?php
    if (isset($_GET['nomPers'])) //Tester si $_GET['nomPers'] est défini ?
    {
      if (!empty($_GET['nomPers'])) //Tester si $_GET['nomPers'] est non vide
      {
        echo 'Bonjour ' . $_GET['nomPers'] . "
";
      }
      else
      {
        echo 'Aucune valeur saisie';
      }
      else
      {
        echo 'Utilisation incorrecte';
      }
    }
  ?>
</body>
</html>
```

Master I/EE

54

Formulaires contenant des champs « SELECT »

← → 🌐 🌐 localhost/monsite/form2.html

Choisissez une couleur :

Blanc ▼	envoyer
Blanc	
Vert	
Bleu	
Rouge	
Noir	

← → 🌐 🌐 localhost/monsite/traiterForm2.php?couleur=Bleu

Vous avez choisi la couleur : Bleu

Master IESE

55

Formulaires contenant des champs « SELECT unique »

```
<html>
<head> <title>Formulaire de saisie des couleurs</title></head>
<body>
  <form action="traiterForm2.php" method="get">
    Choisissez une couleur&nbsp;:
    <select name="couleur">
      <option value="Blanc">Blanc</option>
      <option value="Vert">Vert</option>
      <option value="Bleu">Bleu</option>
      <option value="Rouge">Rouge</option>
      <option value="Noir">Noir</option>
    </select>
    <input type="submit" value="envoyer">
  </form>
</body>
```

</html>
Master IESE

56

Exemple 2 – Traitement en PHP

```
<html>
<head><title>Liste de couleurs</title></head>
<body>
  <?php
    if (isset($_GET['couleur']))
    {
      /* La variable $_GET['couleur'] est définie */
      echo "<p>Vous avez choisi la couleur :". $_GET['couleur'] . "\n";
    }
    else
      echo "<p>Vous n'avez pas choisi de couleur\n";

  ?>
</body>
</html>
```

Formulaires contenant des champs « SELECT multiple »

← → 🔍 localhost/monsite/form3.html

Choisissez des couleurs :

Blanc	↑
Vert	
Bleu	
Rouge	▼

envoyer

← → 🔍 localhost/monsite/traiterForm3.php?couleurs%5B%5D=Blanc&couleurs%5B%5D=Bleu

Vous avez choisi : Blanc

Vous avez choisi : Bleu

Codes des []

Formulaires contenant des champs « SELECT multiple »

```
<html>
<head> <title>Formulaire de saisie des couleurs</title></head>
<body>
  <form action="traiterForm2.php" method="get">
    Choisissez une couleur&nbsp;
    <select name="couleurs[]" multiple>
      <option value="Blanc">Blanc</option>
      <option value="Vert">Vert</option>
      <option value="Bleu">Bleu</option>
      <option value="Rouge">Rouge</option>
      <option value="Noir">Noir</option>
    </select>
    <input type="submit" value="envoyer">
  </form>
</body>
</html>
```

Master I/EE

59

Attention: Si pas de « [] », alors la variable couleurs va être traitée par PHP en tant que variable simple et non pas un tableau. Résultat: la variable couleurs va contenir le dernier choix au niveau du formulaire.

Exemple 3 – Traitement en PHP

```
<html>
<head> <title>Liste de couleurs</title></head>
<body>
  <?php
    if (isset($_GET['couleurs']) && is_array($_GET['couleurs']))
    { /* La variable $_GET['couleurs'] est définie et elle est un tableau */
      foreach($_GET['couleurs'] as $couleur)
        echo "<p>Vous avez choisi : " . $couleur . "n";
      }
    else
      echo "<p>Vous n'avez pas choisi de couleurn";
  }
?>
</body>
</html>
```

Master I/EE

60

Formulaires contenant des champs « CHECKBOX »

← → ↻ localhost/monsite/form4.html

Choisissez des couleurs :

☒ Blanc

☐ Vert

☒ Bleu

☒ Rouge

☐ Noir

← ↻ localhost/monsite/form4.html?couleurs%3F%3D-Blanc&couleurs%3F%3D-Bleu&couleurs%3F%3D-Rouge

Vous avez choisi : Blanc

Vous avez choisi : Bleu

Vous avez choisi : Rouge

Formulaires contenant des champs « CHECKBOX »

```
<html>
<head><title>Formulaire de saisie des couleurs</title></head>
<body>
  <form name="formu" action="traiterForm4.php" method="get">
    Choisissez des couleurs&nbsp;:
    <p><input type="checkbox" name="couleurs[]" value="Blanc">Blanc
    <p><input type="checkbox" name="couleurs[]" value="Vert" >Vert
    <p><input type="checkbox" name="couleurs[]" value="Bleu" >Bleu
    <p><input type="checkbox" name="couleurs[]" value="Rouge">Rouge
    <p><input type="checkbox" name="couleurs[]" value="Noir">Noir
    <p><input type="submit" value="Envoyer">
  </form>
</body>
</html>
```


Exemple 4 – Traitement en PHP

```
<html>
<head><title>Liste de couleurs</title></head>
<body>
  <?php
    if (isset($_GET['couleurs']) && is_array($_GET['couleurs']))
    { /* La variable $_GET['couleurs'] est définie et elle est un tableau */
      foreach($_GET['couleurs'] as $couleur)
        echo "<p>Vous avez choisi : " . $couleur . "\n";
      }
    else
      echo "<p>Vous n'avez pas choisi de couleur\n";
  }
?>
</body>
</html>
```

Master/IESE

63

Formulaires contenant des champs « RADIO »

← → ↻ 🌐 localhost/monsite/form5.html

Choisissez des couleurs :

☐ Blanc

☐ Vert

☒ Bleu

☐ Rouge

☐ Noir

[Envoyer]

Vous n'avez pas choisi de couleur

← → ↻ 🌐 localhost/monsite/form5.html

Choisissez des couleurs :

☐ Blanc

☐ Vert

☒ Bleu

☐ Rouge

☐ Noir

[Envoyer]

Vous avez choisi la couleur : Bleu

Master/IESE

64

Formulaires contenant des champs « RADIO » avec un choix par défaut

```
<html>
<head><title>Formulaire de saisie des couleurs</title></head>
<body>
  <form name="formu" action="traiterForm5.php" method="get">
    Choisissez des couleurs&nbsp;:
    <p><input type="radio" name="couleur" value="Blanc" checked>Blanc
    <p><input type="radio" name="couleur" value="Vert" >Vert
    <p><input type="radio" name="couleur" value="Bleu" >Bleu
    <p><input type="radio" name="couleur" value="Rouge">Rouge
    <p><input type="radio" name="couleur" value="Noir">Noir
    <p><input type="submit" value="Envoyer">
  </form>
</body>
</html>
```

Master I/SE

67

Exemple 6 - Formulaires contenant des champs « RADIO » avec un choix par défaut

← → Ⓞ localhost/monsite/form5.html

Choisissez des couleurs :

☒ Blanc

☐ Vert

☐ Bleu

☐ Rouge

☐ Noir

Envoyer

Vous avez choisi la couleur :Blanc

Master I/SE

68

Formulaires contenant des champs « RADIO »

```
<html>
<head><title>Formulaire de saisie des couleurs</title></head>
<body>
    <form name="formu" action="traiterForm5.php" method="get">
        Choisissez des couleurs&nbsp;:
        <p><input type="radio" name="couleur" value="Blanc">Blanc
        <p><input type="radio" name="couleur" value="Vert" >Vert
        <p><input type="radio" name="couleur" value="Bleu" >Bleu
        <p><input type="radio" name="couleur" value="Rouge">Rouge
        <p><input type="radio" name="couleur" value="Noir">Noir
        <p><input type="submit" value="Envoyer">
    </form>
</body>
</html>
```

Master IEESE

65

Exemple 5 – Traitement en PHP

```
<html>
<head><title>Liste de couleurs</title></head>
<body>
    <?php
        if (isset($_GET['couleur']))
        { /* La variable $_GET['couleur'] est définie */
            echo "<p>Vous avez choisi la couleur :". $_GET['couleur']. "\n";
        }
        else
            echo "<p>Vous n'avez pas choisi de couleur\n";
    ?>
</body>
</html>
```

Master IEESE

66

SGBD et PHP

PHP et Bases de Données

- PHP peut interagir avec des bases de données (BD) pour produire des pages web dynamiques
- PHP peut être utilisé comme interface pour interroger des BD gérées par un SGBD comme:

Access

MySQL

SQL server

Oracle, etc.

MySQL est le plus utilisé avec PHP:

Multi-plates-formes: Windows, UNIX, Linux, Mac OS

Présent chez de nombreux hébergeurs

Gratuit et Open source

Simple à installer et à utiliser (Requêtes SQL)

Performant : rapide, scalable, sécurisé et stable

Gestion puissante des privilèges et des droits d'accès aux BDs

Possède l'outil de gestion puissant phpMyAdmin

MySQL : Requêtes SQL

- Création/suppression et modification des tables
CREATE, DROP, ALTER
- Sélection/Ajout/modification et suppression des données
SELECT, INSERT, UPDATE, DELETE

Réservées à l'administrateur

- Création de BD : **CREATE DATABASE**
- Attribution des privilèges : **GRANT**

Master ISES

71

PHP et MySQL

Objectifs :

- Création de documents web à partir d'une BD
- Alimentation d'une BD à partir du web
- Mise à jour d'une BD à partir du web
- Suppression dans une BD
- Du côté Serveur MySQL
 - Reçoit la requête SQL
 - Exécute la requête SQL
 - Renvoie le résultat
- Du côté de PHP
 - Extrait les données du client
 - Envoie la requête SQL à MySQL
 - Reçoit le résultat de la requête
- Formate et envoie le résultat au client

Master ISES

72

Interaction entre PHP et MySQL

■ Pour que PHP puisse effectuer des requêtes vers MySQL il faut:

- 1- Etablir la connexion à MySQL et choix de la base de données
- 2- Exécuter une requête et renvoi du résultat
- 3- Formater le résultat sous forme de tableau ou objet (Récupérer et parcourir le résultat de la requête)
- 4- Fermer la connexion

■ PHP dispose deux extensions permettant d'effectuer toutes ces opérations (sans utilisation de phpMyAdmin):

L'extension MySQLi

- Fonctions améliorées d'accès à MySQL
 - Proposent plus de fonctionnalités et sont plus à jour
- L'extension PDO (PHP Data Objects)
- Extension orientée objet de PHP
 - Outil complet permettant d'accéder à n'importe quel type de base de données: MySQL, PostgreSQL, Oracle, etc.

Interaction entre PHP et MySQL

■ Etablissement de Connexion à MySQL et choix de la base de données

```
$liendb = new mysqli (serveur, login, mot_passe, nom_base );
```

■ Exécution d'une requête et renvoi du résultat

```
$resultat = $liendb->query($requete);
```

■ Formatage du résultat:

```
    sous forme d'un tableau simple:           $ligne=$resultat->fetch_row();
```

```
    sous forme d'un tableau associatif:        $ligne=$resultat->fetch_assoc();
```

```
    sous forme d'un objet:                     $ligne=$resultat->fetch_obj();
```

■ Fermeture de connexion

```
    $liendb->close();
```


Interaction entre PHP et MySQL : Exemple

Une seule ligne dans le résultat de la requête:

```
<?php
$lien = new mysqli ("localhost","root","","bd_produits" );
$resultat = $lien->query("SELECT * FROM produit where ref=1");
$prod = $resultat->fetch_assoc();
$nom = $prod["nom"];
$prix = $prod["prix"];
echo "le produit rechercher est ".$nom . " son prix est : " . $prix . "DH\n";
$lien->close();
?>
```

Formatage sous forme d'un tableau associatif

Master/IESE

75

Interaction entre PHP et MySQL : Exemple

Une seule ligne dans le résultat de la requête:

```
<?php
$lien = new mysqli ("localhost","root","","bd_produits" );
$resultat = $lien->query("SELECT * FROM produit where ref=1");
$prod= $resultat->fetch_row();
$nom = $prod[0];
$prix = $prod[1];
echo "le produit rechercher est ".$nom . " son prix est : " . $prix . "DH\n";
$lien->close();
?>
```

Formatage sous forme d'un tableau simple

Master/IESE

76

Interaction entre PHP et MySQL : Exemple

Une seule ligne dans le résultat de la requête:

```
<?php
```

```
$lien = new mysqli ("localhost","root","","bd_produits" );
```

```
$resultat = $lien->query("SELECT * FROM produit where ref=1");
```

```
$prod = $resultat->fetch_object();
```

Formatage sous forme d'un objet

```
$nom = $prod->nom;
```

```
$prix = $prod->prix;
```

```
echo "le produit rechercher est ".$nom . " son prix est : " . $prix . "DH\n";
```

```
$lien->close();
```

```
?>
```

Master I/ESF

77

Teste des cas particuliers: Exemple

```
<?php
```

```
$lien = new mysqli ("localhost","root","","bd_produits" );
```

```
// vérifier la connexion
```

```
if ($lien->connect_errno) {
```

```
    echo "Problème de connexion à MySQL: " . $lien->connect_error;
```

```
    exit();
```

```
}
```

```
$resultat = $lien->query("SELECT * FROM produit where ref=1");
```

```
$prod = $resultat->fetch_assoc();
```

```
if($prod){ //vérifier le résultat de la requête
```

```
    $nom = $prod["nom"];
```

```
    $prix = $prod["prix"];
```

```
    echo "le produit rechercher est ".$nom . " son prix est : " . $prix . "DH\n";
```

```
}
```

```
else
```

```
    echo "Produit n'existe pas !\n";
```

```
$lien->close();
```

```
?>
```

Master I/ESF

78

PHP et MySQL - Fonctions d'exploitation du résultat

Plusieurs lignes dans le résultat:

```
mysql_num_rows($resultat)
```

- retourne le nombre de lignes du résultat de la requête

```
mysql_num_fields($resultat)
```

- retourne le nombre de champs du résultat de la requête

- utiliser pour parcourir une ligne du résultat de la requête (cas Formatage sous forme d'un tableau simple)

Plusieurs appels de la fonction:

- `$resultat->fetch_assoc();` (ou bien `$resultat->fetch_row()` ou `$resultat->fetch_object()`);

- A chaque appel le pointeur du résultat passe à la ligne suivante

Master/ISE

79

Interaction entre PHP et MySQL : Exemple

Plusieurs lignes dans le résultat de la requête:

```
<?php
```

```
$lien = new mysqli ("localhost", "root", "", "bd_products");
```

```
$resultat = $lien->query("SELECT * FROM produit");
```

```
$nb lignes = $resultat->num_rows;
```

```
for($i=0;$i<$nb lignes;$i++) {
```

```
    $prod = $resultat->fetch_assoc(); //Cas de tableau associatif
```

```
    $nom = $prod["nom"];
```

```
    $prix = $prod["prix"];
```

```
    echo "<p>". $nom . " son prix est : " . $prix . "DH\n";
```

```
}
```

```
$lien->close();
```

```
?>
```

Master/ISE

80

Affichage amélioré sous forme d'un tableau : Exemple

```

? Plusieurs lignes dans le résultat de la requête:
<?php
    $lien = new mysqli ("localhost", "root", "", "bd_produits" );
    // vérifier la connexion
    $resultat = $lien->query("SELECT * FROM produit");
    $nb lignes = $resultat->num_rows;
    echo "<table border='1' cellspacing='0' cellpadding='5'>";
    echo "<tr bgcolor='gray'><td>Réf</td><td>Désignation</td><td>Prix (DH)</td></tr>\n";
    for ($i=0; $i<$nb lignes; $i++){
        $sprod = $resultat->fetch_assoc();
        $ref = $sprod["ref"];
        $nom = $sprod["nom"];
        $prix = $sprod["prix"];
        echo "<tr><td>$ref</td><td>$nom</td><td>$prix</td></tr>";
    }
    echo "</table>";
    $lien->close();

```

Master IJEE

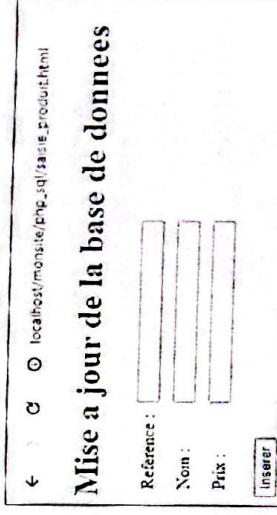
81

PHP et MySQL : Utilisation des formulaires

```

<html>
<body>
<h1> Mise a jour de la base de donnees</h1>
<form method="post" action="maj.php">
    <table cellpadding='5'>
        <tr><td>Reference :</td><td><input type="text" name="ref"></td></tr>
        <tr><td>Nom :</td><td><input type="text" name="nom"></td></tr>
        <tr><td>Prix :</td><td><input type="text" name="prix"></td></tr>
    </table>
    <p><input type=submit value="Insérer">
</form>
</body>
</html>

```



Master IJEE

82

PHP et MySQL : Utilisation des formulaires - Exemple

```
<html>
<body>
<h1> Mise A Jour </h1>
<?php
    $lien = new mysqli ("localhost","root","","bd_produits" );
    // récupération des valeurs du formulaire
    $ref = $_POST['ref'];
    $nom = $_POST['nom'];
    $prix= $_POST['prix'];
    // insertion des valeurs dans la base
    $requete = "INSERT INTO produit (ref,nom,prix) VALUES ('$ref','$nom','$prix')";
    $resultat = $lien->query($requete);
    $lien->close();
?>
</body>
</html>
Mhs/er/EESE
```

83